IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 12 May 2026, accepted 1 June 2026, date of publication 5 June 2026, date of current version 10 June 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3700489

RESEARCH ARTICLE

A Hybrid Reinforcement Learning–Blockchain Architecture for Sequential Decentralised Coordination of Heterogeneous Robots: Proof-of-Concept Validation and Latency Characterisation

ROMMEL VELASTEGUI, RAÚL POLER, AND MANUEL DíAZ-MADROÑERO
Research Centre on Production Management and Engineering (CIGIP), Universitat Politècnica de València, 03801 Alcoy, Alicante, Spain

Corresponding author: Manuel Díaz-Madroñero (fcodiama@cigip.upv.es)

This work was supported by the Horizon Europe Framework Programme (HORIZON) through "Agile Manufacturing as a Service through AI Autonomous Agents" (MaaSAI) under Grant 101177368.

ABSTRACT Contemporary manufacturing systems exhibit structural limitations arising from centralised architectures, which constrain their resilience, scalability, and operational transparency. The reliance on central controllers in multi-agent robotic environments creates single points of failure, hinders immutable traceability, and obstructs the independent auditing of critical processes, thereby limiting the feasibility of truly autonomous and self-orchestrating systems demanded by emerging Industry 4.0 and 5.0 paradigms. This study introduces a conceptual framework for blockchain-enabled autonomous manufacturing, integrating decentralised applications, smart contracts, and distributed robotic agents. We propose and experimentally evaluate a three-layer decentralised architecture integrating tabular Q-learning and Ethereum smart contracts for heterogeneous multi-robot coordination in manufacturing environments. The architecture eliminates centralised control servers by routing coordination signals through blockchain-verified events, enabling immutable task logging and sequential verifiable agent synchronisation in a two-agent laboratory configuration. An experimental prototype comprising a KUKA youBot mobile agent—trained via Q-learning over a four-state, four-action navigation policy—and a UFactory Lite 6 industrial manipulator was deployed in a laboratory environment under ROS Noetic. Quantitative evaluation demonstrates that the Q-learning agent converges to a 96% success rate within approximately 600 training episodes, and that blockchain-mediated coordination introduces a mean end-to-end latency of 45.7 s (blockchain overhead: ∼15 s), acceptable for supervisory task orchestration but incompatible with real-time control loops. These results provide a reproducible proof-of-concept baseline for two-agent sequential blockchain-RL coordination under controlled laboratory conditions, establishing a foundation for future scalability analysis and identify scalability, concurrency, and fault tolerance as critical directions for future work.

INDEX TERMS Blockchain, multi-agent robotic systems, industry 4.0, industry 5.0, DApp, manufacturing orchestration.

I. INTRODUCTION

The coordination of heterogeneous robots—mobile platforms and manipulators—poses a fundamental challenge in manufacturing: achieving local autonomy while maintaining global coherence without a central server [1]. Centralised control architectures introduce single points of failure and constrain scalability and robustness in tightly coupled multi-robot operations [2]. Although decentralised approaches exist, they largely fail to integrate reinforcement learning with verifiable, distributed validation mechanisms that ensure consistent and trustworthy coordination [3]. As a

The associate editor coordinating the review of this manuscript and approving it for publication was Nafees Mansoor.

result, reliable serverless coordination among heterogeneous robotic agents remains an open problem. This paper does not aim to advance reinforcement learning theory; rather, it introduces and experimentally validates a verifiable cyber-physical coordination (CPC) architecture that integrates decentralised applications (DApps), Ethereum smart contracts, and real-time ROS middleware. The primary contributions concern: (a) verifiable traceability of autonomous decisions; (b) on-chain auditing of robotic actions; (c) robot–blockchain integration under measurable latency constraints; and (d) a reproducible architectural baseline for decentralised multi-agent coordination. Unlike conventional robotic middleware or message-passing systems, the proposed architecture provides immutability, public deterministic execution, non-repudiation of autonomous decisions, and coordination without a trusted central authority—properties that are increasingly required in multi-stakeholder industrial environments, autonomous supply chains, and AI-regulated robotic systems [4].

There is a clear research gap in the absence of experimentally validated models that combine local reinforcement learning with distributed blockchain mechanisms for the coordination of heterogeneous robots [5]. Existing studies typically address reinforcement learning or blockchain-based decentralisation in isolation, without demonstrating their joint operation under real-time, multi-robot manufacturing constraints. Consequently, the feasibility and performance of serverless coordination architectures integrating both elements remain insufficiently validated [2].

Existing work on blockchain technology in manufacturing concentrates on supply chain management and traceability, rather than on direct operational coordination among robotic systems [6]. In contrast, research on multi-agent robotic systems typically assumes centralised or semi-centralised control, leaving fully decentralised coordination architectures largely unaddressed. Table 1 summarises the key trust and network properties that differentiate the technologies evaluated in this study.

TABLE 1. Comparison of technologies with model and network type.

Technology	Trust Model	Network Type
MQTT	Central broker	Centralised
DDS (ROS 2)	Distributed middleware, no immutable consensus	Federated
Hyperledger Fabric	Permissioned, closed governance	Private
Ethereum	Trust-minimised, public verification	Open

The goal of this work is not to minimise latency, but to explore trust-minimised coordination in environments where no single stakeholder can be assumed to be fully trusted. Blockchain introduces governance and verifiability properties into autonomous physical systems that cannot be replicated by equivalent message-passing solutions such as ROS with MongoDB [7].

This study is structured around three specific and testable research questions addressing not the algorithmic efficiency of RL, but the formal behaviour of learning under distributed consensus-induced delay, and the viability of trust-minimised coordination in physical multi-agent systems: RQ1: Can blockchain operate as an activation and distributed memory layer in a multi-robot system without interfering with a reinforcement learning–based control loop? RQ2: Can a Q-learning mobile agent, trained with adapted sensors in an academic environment, coordinate safely with an industrial manipulator without centralised planning? RQ3: Is the combination of local autonomy through reinforcement learning and distributed verification via blockchain viable while maintaining acceptable operational latencies in manufacturing contexts?

The contributions of this work are fourfold: (i) a verifiable CPC architecture integrating DApps, smart contracts, and ROS middleware; (ii) its physical experimental demonstration under real consensus latency constraints in a two-agent sequential configuration; (iii) quantitative characterisation of blockchain-induced delay impact on RL convergence dynamics; and (iv) empirical evidence that functional equivalence between blockchain and conventional middleware does not imply property equivalence. Full elaboration of each contribution is provided in the corresponding sections. The present work extends the conceptual framework of [1] in three substantive respects: (a) it advances from architectural specification to physical experimental validation with real robotic hardware; (b) it introduces quantitative characterisation of blockchain consensus latency impact on RL convergence dynamics, a relationship left unanalysed in prior work; and (c) it provides the first empirical comparison between blockchain-mediated and non-blockchain coordination baselines within this architectural family, establishing quantitative performance bounds absent from prior conceptual contributions.

The objective of this work is not to fully close the identified research gaps in a theoretical sense, but to provide an experimentally validated architectural baseline upon which formal, scalable, and blockchain-agnostic coordination models can be constructed. Whilst recent work (2023–2024) has explored blockchain-enabled industrial coordination, most studies remain at simulation level or abstract architectural discussion; the present contribution differentiates itself by providing physical robotic validation under ROS with real consensus latency constraints [7]. Establishing a reproducible experimental baseline for decentralised multi-robot manufacturing systems.

This article is structured following a coherent progression that supports the development and validation of the proposed approach. Section I introduces the research problem, objectives, and main contributions. Section II reviews the literature on multi-agent robotic systems (MARS), blockchain technology (BCT) in industrial environments, smart contracts, and key Industry 4.0 paradigms, identifying existing conceptual and technical gaps. Section III presents the proposed

architecture, detailing its theoretical foundations, structural design, functional components, and system interactions for integrating multi-agent coordination with blockchain in manufacturing systems. Section IV describes the experimental setup, including system configuration, implementation parameters, use-case scenarios, and evaluation metrics. Section V reports the results of the experimental validation through quantitative and comparative analyses. Section VI discusses the findings, examining technical, organisational, and operational implications, as well as limitations and implementation challenges. Finally, Section VII summarises the main conclusions, consolidates the contributions of the study, and outlines directions for future research toward self-orchestrating manufacturing environments enabled by decentralised technologies, followed by the References section.

II. LITERATURE REVIEW

The literature review was conducted using the Web of Science (WoS) database as the primary bibliographic source. The search strategy combined Boolean operators across the core study variables: multi-agent robotic systems, blockchain technology, smart contracts, reinforcement learning, and Industry 4.0 manufacturing. Only peer-reviewed publications written in English were included, ensuring methodological rigour and international scope. The convergence of decentralised digital technologies with physical automation has opened a new frontier in industrial engineering: manufacturing environments where coordination among autonomous agents emerges from shared, verifiable, and tamper-resistant computational infrastructures rather than from centralised command structures. This review traces the scientific trajectory across three interrelated domains before identifying the research gaps that this study addresses.

A. HETEROGENEOUS MULTI-ROBOT COORDINATION

Multi-agent robotic systems (MARS) constitute the conceptual core for modelling environments where multiple autonomous entities collaborate in distributed tasks. Grounded in distributed artificial intelligence, cooperative game theory, and complex adaptive systems, MARS theory describes how physically embodied agents—equipped with perception, reasoning, and actuation capabilities—achieve collective outcomes through local interactions [8]. Three structural properties define these systems: autonomy, enabling local decision-making without centralised supervision; social interaction, providing communicative mechanisms for negotiation and conflict resolution among heterogeneous agents; and adaptive reactivity, allowing dynamic behavioural modification in response to environmental changes or shifting objectives [9].

Classical coordination relied on centralised planners that orchestrated task assignment and conflict resolution from a single computational node. As manufacturing systems incorporated heterogeneous agents with different capabilities and operational tempos, the limitations of this paradigm became critical: single points of failure, exponential scaling costs, and opaque decision rationales [10]. Agent coordination architectures evolved along a spectrum from hierarchical—offering predictable control at the cost of bottlenecks—to heterarchical—distributing decision-making fully at the cost of coordination complexity—with hybrid approaches seeking intermediate balances.

Decentralised alternatives—swarm intelligence, game-theoretic allocation, and reinforcement learning—demonstrated that effective collective behaviour could emerge from local rules without global supervision. However, classical consensus methods such as leader election, auction-based allocation, and distributed voting function effectively only in closed cooperative environments, offering limited guarantees where agents may be unreliable or subject to external manipulation. Recent advances in multiagent consensus tracking over asynchronous cooperation–competition networks [11] and convergence analysis of products of substochastic matrices under communication delays [12] provide theoretical foundations for characterising coordination stability under bounded asynchronous delays. Nevertheless, a persistent structural deficit remained: the absence of any mechanism for maintaining verifiable and globally consistent records of autonomous decisions, making trust between agents dependent on assumptions that could not be cryptographically guaranteed. This void motivated the exploration of blockchain technology as a coordination substrate capable of transforming direct agent-to-agent negotiation into indirect coordination through distributed, immutable, and auditable records.

B. BLOCKCHAIN TECHNOLOGY AS A COORDINATION SUBSTRATE FOR INDUSTRIAL ROBOTICS

Blockchain technology (BCT) constitutes a distributed computational infrastructure enabling multiple entities to maintain reliable consensus over a shared dataset without central authority. Three formal properties underpin its relevance for industrial coordination. Cryptographic immutability is achieved through chained data structures where each block contains a hash of its predecessor, making retroactive modification computationally infeasible and guaranteeing temporal integrity of operational records. Decentralised validation distributes verification and storage across independent nodes, each maintaining a complete state replica, thereby eliminating single points of failure. Distributed consensus protocols—Proof of Stake (PoS), Proof of Authority (PoA), Byzantine Fault Tolerance (BFT)—define the procedural rules by which nodes agree on valid transactions and their ordering [13]. The adoption of BCT in industrial contexts progressed from passive record-keeping—supply chain logging, quality auditing, maintenance traceability—towards active coordination. The decisive conceptual advance was recognising that smart contracts—deterministic, self-executing programs deployed on the blockchain—could encode not merely records of past events but executable rules for future

interactions: conditions for initiating production sequences, resource allocation policies, quality validation protocols, or reconfiguration triggers. These programs allow operational logic to be enforced automatically, transparently, and without the possibility of unilateral modification by any single participant [3]. On-chain events generated by smart contracts serve as asynchronous signalling channels to which robotic agents subscribe, triggering physical actions in response to verified state changes on the distributed ledger. This mechanism extends the perception-decision-action cycle of each robot to include a blockchain-mediated verification step, ensuring that every action is not only locally rational but also globally consistent with the rules agreed upon by all participants. Decentralised applications (DApps) complement this architecture by providing the human-system interface, enabling operators to issue commands, adjust parameters, and audit records within a transparent and traceable infrastructure [14].

Despite this conceptual promise, significant technical barriers persist. Consensus-induced latency conflicts with real-time robotic constraints; transaction throughput limits the volume of coordinated interactions; energy consumption remains a consideration for sustainable deployment; and interoperability between blockchain platforms and industrial control systems is underdeveloped [14]. Consequently, most implementations are restricted to simulations or low-criticality scenarios, with physical demonstrations involving heterogeneous robots under blockchain-mediated verification being remarkably scarce.

C. REINFORCEMENT LEARNING IN ROBOTIC NAVIGATION

Reinforcement learning (RL) has matured into a core framework for autonomous robotic decision-making. Classical tabular methods such as Q-learning and deep variants including Deep Q-Networks (DQN) enable agents to learn navigation and control policies directly from sensory inputs—LIDAR, vision, force-torque—without explicit dynamic models [15]. These techniques have been successfully applied to obstacle avoidance, goal-directed movement, and adaptive path planning in environments characterised by uncertainty and partial observability.

Yet the vast majority of RL-based robotic systems operate under a critical simplification: each robot learns and acts as if it were the sole agent in its environment, or coordinates through centralised planners and predefined communication protocols rather than through verifiable distributed mechanisms. The question of how locally learned policies can be reconciled with global coordination requirements—verified, audited, and integrated into a shared operational record—has received insufficient attention. The deliberate use of tabular Q-learning with a reduced, discrete state space in the present work serves an explicit methodological purpose: isolating the effect of blockchain-mediated coordination delay on learning dynamics. A simpler policy enables direct measurement of the impact of asynchronous, consensus-induced latency on convergence behaviour—an effect that would be confounded by the training complexity of deep architectures. The value of the RL component in this architecture does not reside in algorithmic sophistication per se, but in three properties that are particularly relevant when embedded within a consensus-driven coordination layer: (a) the environment is partially observable, introducing genuine decision uncertainty; (b) dynamic obstacles generate non-stationary transitions that cannot be fully modelled a priori; and (c) blockchain-induced latency introduces non-deterministic asynchrony into the perception–decision–action cycle, making the temporal relationship between observed state and executed action inherently variable.

Research at the RL-blockchain intersection remains fragmentary. Existing proposals use blockchain as a reward-accounting or incentive mechanism in abstract multi-agent simulations, but none have demonstrated the coexistence of a local RL control loop with a distributed blockchain verification layer without degrading learning stability, reaction time, or safety [6]. This absence constitutes a critical barrier to deploying truly autonomous and verifiable manufacturing systems.

D. RESEARCH GAPS

Whilst the authors' prior work [1, 11, 19] established the conceptual architecture and theoretical motivation for blockchain-mediated multi-agent robotic coordination, those contributions lacked physical experimental validation, quantitative latency characterisation, and empirical baseline comparison—the three elements that differentiate the present contribution from the prior body of work.

Four specific gaps crystallise from this analysis. First, no consolidated architectural model specifies how operational commands, autonomous decision-making, and distributed action recording coexist within a single decentralised system; existing proposals address these elements in isolation, limiting reproducibility and scalability [1]. Second, the impact of blockchain validation latency on robotic coordination remains unresolved: current approaches either avoid tight coupling between blockchain and control loops or sacrifice decentralisation through off-chain intermediaries that reintroduce centralised trust assumptions [16]. Third, interoperability between blockchain platforms and heterogeneous robotic hardware lacks shared data models and standardised interaction patterns. Fourth, empirical validation involving physical robots performing coordinated tasks under decentralised verification constraints is critically scarce, leaving robustness, fault tolerance, and real-world performance insufficiently characterised [17]. This study positions itself explicitly within these gaps by proposing and experimentally validating a decentralised architecture for autonomous manufacturing, in which heterogeneous robotic agents coordinate through blockchain-mediated smart contracts under real operational constraints. The aim is not merely to demonstrate technical feasibility but to provide an integrative framework that bridges the disciplinary boundaries identified in this

review, addressing the architectural, latency, interoperability, and empirical validation deficits that currently limit the field.

III. PROPOSED ARCHITECTURE

This section presents the proposed hybrid architecture, which decouples local autonomy based on reinforcement learning from distributed verification via smart contracts. The architecture is organised in three interconnected layers as shown in Figure 1.

A. HYBRID FRAMEWORK OVERVIEW

The conceptual model is organised as a three-layer interconnected architecture intended to support decentralised coordination among multi-agent robotic systems using blockchain-based infrastructure. Rather than relying on a central control server, coordination logic and state information are distributed across blockchain components and interacting agents [13].

Layer 1: Decentralised Interface serves as the human–system interaction layer through decentralised applications (DApps). This layer enables users to issue operational commands that are encoded as blockchain transactions. Each command is digitally signed and submitted to the network, providing a consistent mechanism for registering user intentions and system inputs without direct interaction with the robotic control layer. Layer 2: Blockchain Interaction constitutes the coordination and validation layer. Smart contracts deployed on a public or consortium blockchain receive transactions originating from the DApps, evaluate predefined conditions, and update the shared system state accordingly. When relevant conditions are satisfied, these contracts emit on-chain events that represent coordination signals. The smart contracts encode operational rules related to task sequencing, resource availability, or execution constraints, and their execution is replicated across validating nodes to maintain a consistent global state. Layer 3: Physical Execution comprises autonomous robotic agents that act as blockchain-connected nodes. Each agent continuously monitors the blockchain for events relevant to its functional role and translates these digital signals into physical actions. Local control and optimisation remain onboard the robot, while coordination with other agents is achieved through shared on-chain state updates rather than direct peer-to-peer control.

Within this architecture, the operational flow follows a DApps → Smart Contract → Robotics sequence. A user action generates a transaction that triggers smart contract logic, resulting in state updates and event emissions. Robotic agents listen for these events, extract the associated parameters, and execute the corresponding tasks. Agents can also submit transactions reporting execution status or exceptional conditions, allowing the blockchain to maintain an updated representation of system-level progress. This bidirectional interaction closes the loop between digital coordination and physical execution and enables the integration of heterogeneous robots and sensors through standardised blockchain events and interfaces.

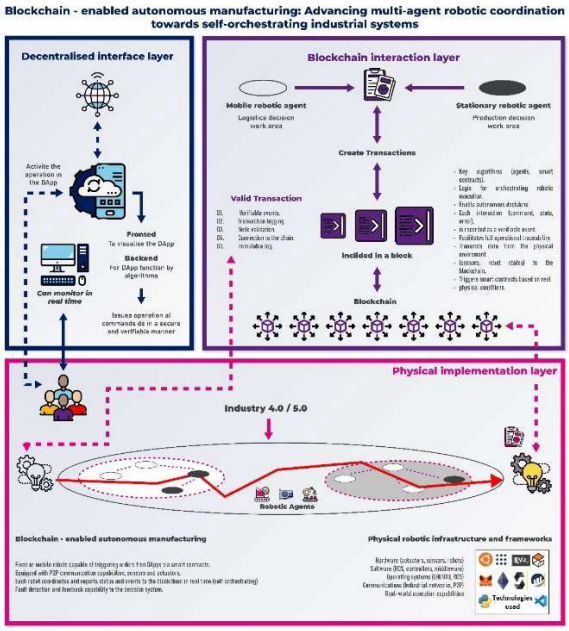

FIGURE 1. Three-layer hybrid architecture: DApp interface, blockchain verification layer, and physical execution layer with RL-based mobile agent and industrial manipulator.

B. LOCAL AUTONOMY LAYER: Q-LEARNING MOBILE AGENT

The mobile agent is implemented as an autonomous navigation robot controlled by a discrete Q-learning algorithm operating entirely onboard. The state space is defined as $s_t = [odom.alert_t, scanner.alert_t]$, where odom.alert indicates deviation or instability in odometry, and scanner.alert denotes obstacle detection based on LIDAR distance thresholds. This abstraction yields four discrete states. The action space is A = Forward, Left, Right, Wait, corresponding to linear motion, angular correction, or stopping. Action selection follows an ε-greedy policy over a tabular Q-function.

Tabular Q-learning was selected over deep reinforcement learning alternatives (DQN, PPO) based on three explicit methodological criteria relevant to the research objective. First, the state space defined by {odom.alert, scanner.alert} is discrete and four-dimensional, rendering function approximation unnecessary and introducing avoidable training variance. Second, tabular Q-learning provides a transparent, analytically tractable policy whose convergence under bounded asynchronous delay can be formally characterised—a property critical for isolating the effect of blockchain-induced latency on learning dynamics, which would be confounded by the stochastic gradient dynamics of deep architectures. Third, the tabular formulation enables direct measurement of the τ-delay effect on Q-value updates as described in Section III-G. The Baseline 3 comparison with a lightweight DQN (two fully connected layers, 64 units, ReLU activations) reported in Section V demonstrates that, for the defined discrete state space, tabular Q-learning

achieves equivalent or superior convergence speed with substantially lower computational overhead.

Regarding the reward signal, the reward function combines kinematic and safety objectives through a weighted sub-component formulation. The relative contribution of each sub-component is governed by scalar weights w1 and w2, assigned values of 0.6 and 0.4 respectively, reflecting the deliberate prioritisation of goal-directed navigation over obstacle avoidance in an environment where collisions are penalised terminally rather than through graduated discouragement. These weights were determined empirically through a grid search over the set {0.3, 0.4, 0.5, 0.6, 0.7} for each parameter across 200-episode pilot runs, selecting the combination that maximised convergence speed whilst maintaining a collision rate below 5% during the exploratory phase. Full numerical specification of all reward sub-components is provided in Section IV-C. Q-values are updated using the standard temporal-difference rule: $Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_t + \gamma \text{amax} Q(s_{t+1}, a) - Q(s_t, a_t)]$

C. INDUSTRIAL MANIPULATOR COORDINATION

The industrial agent is a UFactory Lite 6 robotic manipulator with six revolute joints and one gripper degree of freedom, resulting in seven joint configurations. The end-effector is a vacuum gripper used for pick-and-place tasks.

The manipulator operates in two modes: simulation mode, using a ROS-based physics simulator, and real mode, executing identical commands on physical hardware via ROS drivers. The manipulator does not perform learning or planning; instead, it executes predefined joint-space trajectories triggered by external coordination events.

D. DISTRIBUTED VERIFICATION LAYER: SMART CONTRACT DESIGN

Each robotic agent incorporates a lightweight blockchain interface operating as a light node, responsible for subscribing to specific smart contract events and submitting signed transactions when required. The smart contract was intentionally designed as a minimal coordination primitive to isolate the impact of consensus delay and event-triggered synchronization on robotic learning dynamics. This design choice must not be interpreted as a simplification of the contract's functional role. Unlike a REST endpoint or a ROS topic, the smart contract provides: (a) immutable recording of every coordination decision as a cryptographically verifiable on-chain event; (b) deterministic public execution of coordination logic without reliance on any trusted intermediary; (c) non-repudiation of autonomous robotic actions, enabling forensic auditability of decision sequences; and (d) serverless coordination across heterogeneous stakeholders without shared administrative authority. These properties are not merely desirable but increasingly constitutive of regulatory and contractual requirements in industrial AI systems, multi-stakeholder manufacturing networks, and autonomous robotics frameworks subject to emerging AI accountability legislation [4].

Coordination is implemented through a single smart contract exposing an issueCommand() function. When invoked, this function records command parameters and emits a CommandIssued event. This event constitutes the only coordination signal consumed by the robotic agents. No additional orchestration, task allocation, or learning logic is embedded in the contract.

The interaction interface was defined using an ABI that includes the CommandIssued event (with parameters sender indexed by type address and command by type string) for asynchronous notification of issued commands, as well as the issueCommand(string _command) function with nonpayable mutability to record operational instructions in the chain, as shown below in the contract ABI:

```
ABI: # ABI of the contract
abi = [
 {
   "anonymous": False,
   "inputs": [
     {"indexed": True, "internalType": "address",
"name": "sender", "type": "address"},
     {"indexed": False, "internalType": "string", "name":
"command", "type": "string"},
   ],
     "name": "CommandIssued",
     "type": "event",
 },
 {
     "inputs": [{"internalType": "string", "name":
"_command", "type": "string"}],
     "name": "issueCommand",
     "outputs": [],
     "stateMutability": "nonpayable",
     "type": "function",
 },
```

The smart contract operates as a stateless coordination primitive implementing a single-function, single-event interface. Upon invocation of issueCommand(string _command), the contract: (1) validates the transaction signature against the caller's Ethereum address; (2) emits a CommandIssued event encoding the caller address (indexed) and the command string; (3) writes no persistent state, ensuring gas-minimal execution. The Python-based ROS listener subscribes to CommandIssued events via Web3.py's event filter mechanism, polling at 5-second intervals. Upon event detection, the listener extracts the command parameter, validates its format against a predefined command vocabulary {TRAIN, EXECUTE, STOP}, and publishes the corresponding ROS action to the /robot_command topic. This two-stage validation—on-chain signature verification followed by off-chain command validation—ensures that only cryptographically authenticated and semantically valid commands trigger physical robotic actions.

Ethereum was selected not for latency optimality but for its adversarial consensus model and deterministic smart contract execution under public verification constraints. During local development, a single-node Ganache instance was employed for deterministic, controlled experimentation. Public testnet validation relied on the distributed validator infrastructure of the Ethereum Sepolia network, providing real consensus behaviour under independent validator control. Whilst validator-level control was not within the scope of this study, the focus was on application-layer integration and the characterisation of consensus latency impact on robotic coordination dynamics. Future deployments will incorporate multi-node private Ethereum networks or permissioned blockchain frameworks (e.g., Hyperledger Fabric) to explicitly measure validator independence and Byzantine fault-tolerance behaviour under controlled industrial conditions. All latency figures reported in Section V-B correspond to Sepolia testnet measurements unless otherwie stated.

E. MONITORING INTERFACE

The monitoring interface is implemented as a web based DApp designed exclusively for system observation and visualisation. The frontend is developed in React, providing real-time displays of the mobile robot's estimated position and the evolution of the reinforcement learning reward curve across training episodes. The backend is implemented in Flask, exposing REST endpoints (/api/state/position, /api/state/reward, /api/state/episode) that serve the current system state obtained from the ROS environment. Data are refreshed via client-side polling at a frequency of 1 Hz. Importantly, this monitoring layer is strictly observational and does not intervene in the control loop, learning process, or blockchain-based coordination logic.

F. TEMPORAL FLOW AND DESIGN ASSUMPTIONS

The implemented system follows a sequential and event-driven temporal flow that connects user interaction, blockchain verification, and robotic execution within a single coordinated pipeline. The process begins when a human operator triggers a command through the decentralised application interface, selecting either a training or simulation mode. This action initiates a transaction submitted via the Web3 interface to the blockchain network, where the smart contract executes the corresponding command function and emits a verifiable on-chain event. A Python-based listener, polling the blockchain at five-second intervals, captures this event and translates it into a robotic execution instruction through the ROS middleware. Depending on the command received, ROS launches either the Q-learning training process or the learned policy execution mode, directing the mobile robot to navigate towards the designated target. Upon successful completion of the navigation task, the industrial manipulator executes a predefined pick-and-place trajectory, thereby closing the full coordination cycle from digital command to physical action. Figure 2 presents the UML sequence diagram illustrating this temporal flow across system layers.

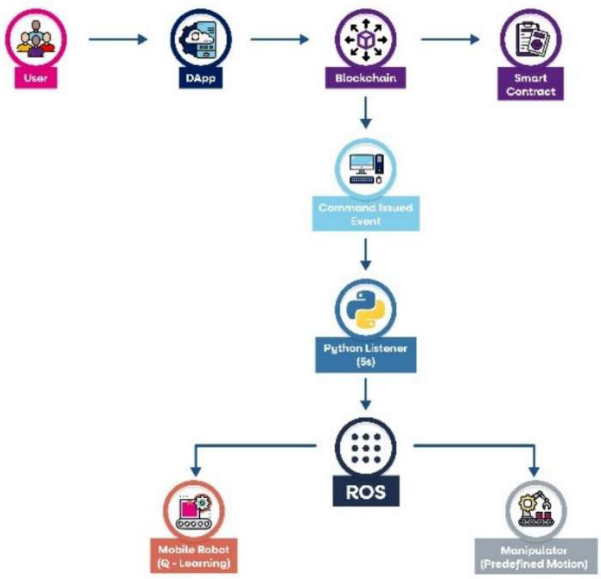

FIGURE 2. Sequence diagram (UML sequence diagram or timeline).

The experimental validation was conducted under a deliberately constrained set of design assumptions intended to isolate the core coordination mechanisms from confounding variables. The blockchain network operates locally through Ganache, providing a controlled environment free from the latency variability and throughput limitations of public networks. The event polling interval was fixed at five seconds, establishing a predictable communication cadence between the blockchain layer and the robotic control system. Coordination between agents follows a strictly sequential logic, with no parallel task execution, ensuring that each step in the pipeline completes before the next begins. The physical environment is static and involves only two robotic agents—a mobile robot and an industrial manipulator—operating without fault handling, recovery protocols, or dynamic task reallocation mechanisms. These constraints define the exact operational scope within which the experimental results should be interpreted, while simultaneously establishing a baseline architecture upon which future extensions incorporating concurrency, fault tolerance, and larger agent populations can be systematically evaluated. This section defines the exact operational scope and constraints under which the experimental validation was conducted.

G. FORMAL SYSTEM MODEL UNDER CONSENSUS-INDUCED DELAY

The present work models the hybrid system without full Dec-POMDP formalisation, as the objective is the experimental demonstration of a reliable coordination channel via an immutable consensus layer rather than optimal decision-making under global uncertainty. Future work will formalise the architecture as a Dec-POMDP, modelling the smart contract as a deterministic coordinator with

non-constant latency and providing formal proof of the convergence bounds identified below.

For the purposes of this study, let s_t denote the joint system state, a_t the coordinated action, and $\tau \in [0, \tau\_max]$ the blockchain-induced coordination delay. The effective Q-learning update under asynchronous consensus delay is:

$$Q(s\_t, a\_t) \leftarrow (1-\alpha)Q(s\_t, a\_t) + \alpha[r\_t + \gamma \max\_a Q(s\_\{t+\tau\}, a)]$$

Unlike the standard TD update, the target state is evaluated at $t + \tau$ rather than $t + 1$, introducing a bounded asynchrony into the perception–decision–action cycle that is absent from single-agent formulations.

Proposition (Convergence under bounded delay, informal statement): If $\tau \leq \tau\_max < \infty$ and α satisfies Robbins–Monro conditions, Q-learning under delay τ converges to an ε-neighbourhood of the optimal Q-function, where ε increases monotonically with τ_max. This formulation is consistent with consensus convergence results under communication delay reported in [11] and [12], which establish convergence conditions analogous to the Robbins–Monro requirements identified here.

Note on experimental implementation: The experiments in Section IV employ a fixed learning rate $\alpha = 0.25$, which does not satisfy the Robbins–Monro asymptotic decay condition—a deliberate choice prioritising training speed and reproducibility within the episode budget of N = 1,500. Convergence is therefore assessed empirically through the success-rate plateau reported in Table 6, and formal derivation of the ε-bound as a function of τ_max and α is deferred to future theoretical work alongside the Dec-POMDP formalisation.

IV. EXPERIMENTAL SETUP

The experimental validation was designed to assess the technical feasibility of the proposed architecture under controlled laboratory conditions. This section describes the hardware configuration, software stack, and procedural parameters that defined the physical implementation, providing the reproducible baseline necessary to interpret all subsequent results and to support future comparative studies.

A. HARDWARE CONFIGURATION

The physical experimental environment comprised two heterogeneous robotic agents operating within a laboratory workspace measuring 3 × 5 metres, a constraint imposed by the physical footprint of the facility and by the cable-bound power supply architecture required by the KUKA youBot mobile platform. The mobile agent was a KUKA youBot, a holonomic wheeled robot equipped with built-in proprioceptive odometry and a 2D LIDAR scanner. These two sensor modalities provided the fused state signal consumed by the onboard Q-learning controller: odometry supplied positional drift and instability indicators, whilst the LIDAR scanner provided binary obstacle proximity alerts based on configurable distance thresholds. The industrial agent was a UFactory Lite 6 six-axis collaborative manipulator fitted with a vacuum end-effector, executing predefined pick-and-place joint-space trajectories upon receipt of blockchain-mediated coordination events.

Both agents were interfaced through a dedicated workstation equipped with an Intel Core i7 processor and an NVIDIA Jetson Orin GPU module, which hosted the ROS middleware, the Python-based blockchain event listener, and the Flask backend concurrently. The physical arena incorporated fixed obstacles at predetermined positions as well as dynamically repositioned objects across experimental trials, with multiple start and goal configurations evaluated to assess generalisation. Whilst the constrained cable topology limited the physical footprint, the architecture was designed to scale horizontally within simulation environments, where spatial and power supply restrictions do not apply

B. SOFTWARE STACK

The experimental platform was built using a set of well-established open-source technologies to ensure reproducibility and compatibility with robotic and blockchain environments. The system was deployed on Ubuntu 20.04 as the operating system, providing long-term support and native compatibility with the selected robotic middleware. Robot control and communication were implemented using ROS Noetic, which served as the core middleware for sensor integration, actuation, and inter-process messaging. Simulation and physics-based validation were conducted in Gazebo 11, while real-time robot state visualisation and debugging were supported through RViz 1.14.25-1.

The decentralised coordination layer was implemented on a local blockchain network using Ganache (CLI), enabling controlled experimentation without external network variability. Smart contracts were developed in Solidity 0.8.18 and interacted with the robotic system through Web3.py 7.14.1, which provided programmatic access to blockchain transactions and event monitoring from Python-based ROS nodes. The monitoring subsystem was implemented independently from the control loop. A Flask (Python 3.9) backend exposed REST endpoints for system state inspection, while the frontend was developed in React 19.0.1 to visualise robot position and reinforcement learning metrics. Reinforcement learning was implemented as a custom tabular Q-learning framework, fully integrated within the ROS environment and executed locally on the mobile robot. Table 2 summarises the complete software stack used in the experimental platform.

C. REINFORCEMENT LEARNING CONFIGURATION

In the reinforcement learning environment configuration, three hierarchical levels of interaction were defined in ROS: OdomEnv, responsible for subscribing to the /odom topic to estimate the robot's position and orientation, and for publishing control actions in /cmd_vel; ScannerEnv, responsible for subscribing to the /scan topic and processing LiDAR data segmented into three spatial regions (left, centre, and right)

TABLE 2. Software stack.

Component	Technology	Version
Operating system	Ubuntu	20.04
Robotic middleware	ROS	Noetic
Simulator	Gazebo	11
Visualisation	RViz	1.14.25-1
Blockchain (local)	Ganache	CLI
Smart contracts	Solidity	0.8.18
Blockchain library	Web3.py	7.14.1
Backend monitoring	Flask (Python)	3.9
Frontend monitoring	React	19.0.1
RL framework	In-house implementation	Q-learning tabular

for obstacle detection; and MasterEnv, which integrates both sub-environments by merging the kinematic and perceptual state into a single state vector used by the agent.

The operational parameters of the environment shown in Table 3 are dynamically loaded from a YAML file to facilitate experimental reproducibility. The implemented algorithm corresponds to tabular Q-learning with the following hyperparameters: learning rate $\alpha = 0.25$; discount factor $\Gamma = 0.9$; initial exploration rate $\varepsilon_0 = 0.9$; exploration decay factor ε_decay $= 0.9986$; total number of training episodes N_episodes $= 1500$; maximum steps per episode N_steps $= 150$; and target position defined in the plane as (x_g, y_g) $=$ (6, 2). Table 3 consolidates all Q-learning hyperparameters used throughout the experimental validation.

TABLE 3. Q-learning hyperparameters.

Parameter	Symbol	Value
Learning rate	α	0.25
Discount factor	Γ	0.9
Initial exploration rate	ε₀	0.9
Exploration decay	ε_decay	0.9986
Total training episodes	N_episodes	1500
Max steps per episode	N_steps	150
Goal position	(x_g, y_g)	6 , 2

The scalar reward function introduced in Section III-B is here operationalised as a weighted combination of two sub-components, enabling independent tuning of kinematic and safety objectives, the reward function is formalised as R_((s,a)) = w_1* R_odom (s,a) + w_2* R_scanner (s,a), where R_odom evaluates the kinematic performance with respect to the target, granting a positive reward +r1 when the distance variation Δd is negative (the robot approaches the target) and the angular variation $\Delta\theta$ remains within a threshold θ_max; penalising with −p1 when Δd > 0 (moving away from the target); and assigning a terminal bonus +B when the distance to the target d is less than a threshold d_goal. For its part, capital R subscript s c a. n n e r models navigation safety, granting +r2 when there is no obstacle alert (alert $=$ 1) and the action executed is Forward; penalising with −p2 when there is a risk (alert $= 0$) and Forward is still executed; and applying a severe penalty −P in the event of a collision, ensuring that the agent internalises safety constraints during the learning process

D. BLOCKCHAIN CONFIGURATION

The blockchain configuration of the experimental system was implemented on the Sepolia Testnet, a public testnet of the Ethereum ecosystem that replicates its consensus mechanism under controlled conditions for testing, ensuring decentralisation and distributed validation without actual production costs. Transactions were executed with a gas limit of 500,000 units to ensure sufficient computational capacity during the invocation of smart contract functions deployed at the address 0x13.2E5F442443761BeaB947DCc1891f0621470a7.; The interaction interface was specified using the ABI described in Section III-D, which defines the CommandIssued event and the issueCommand() function governing all robotic coordination signals.

Finally, event detection was performed using a polling mechanism with a 5-second interval, configured as a Web3 listener timeout to periodically query the network and capture on-chain confirmations in a controlled manner.

E. METRICS DEFINITION

All experimental metrics were collected across N $= 5$ independent runs with distinct random initialisations (seed values: 42, 123, 256, 512, 1024). Results are reported as mean ± standard deviation with 95% confidence intervals computed via a t-distribution. Random seeds controlled the ε-greedy exploration sequence, initial obstacle placement (in dynamic configurations), and Q-table initialisation. The following metrics were measured and are summarised in Table 4:

TABLE 4. Experimental metrics.

Metric	Symbol	Unit	Measurement instrument
Blockchain coordination latency	t_bc	s	Web3.py event timestamp
RL navigation success rate	SR	%	ROS episode counter
End-to-end coordination latency	T_total	s	Synchronised system timestamp

To ensure methodological rigour and support causal inference regarding the effect of blockchain integration on coordination behaviour, the experimental protocol was structured as a $2 \times 2 \times 2$ factorial design covering the following factors: 1. Obstacle configuration: static (fixed positions across all trials) versus dynamic (obstacles repositioned between episodes following a predefined randomisation procedure). Five static configurations and five dynamic configurations were evaluated. 2. Blockchain connectivity: enabled (coordination signals routed through Ethereum smart contract)

versus disabled (direct ROS-based command dispatch, constituting the off-chain baseline). 3. Blockchain network: local Ganache (single-node, deterministic) versus Sepolia Testnet (distributed validator infrastructure, real consensus latency). Each cell of the factorial design was evaluated across N = 5 independent runs. Obstacle positions in dynamic configurations were sampled uniformly from a predefined grid of valid non-overlapping placements. The distribution of episodes per condition was balanced across training runs to prevent condition-specific overfitting.

F. EXPERIMENTAL BASELINES

Three experimental baselines were implemented to contextualise the performance of the proposed blockchain-mediated coordination architecture:

Baseline 1 — Centralised ROS coordination: A conventional ROS-based master node was implemented to issue coordination commands directly to the robotic agents via standard topic publishing, without any blockchain layer. This baseline quantifies the latency and success rate achievable without distributed consensus overhead.

Baseline 2 — Direct off-chain communication: Commands were transmitted from the DApp interface directly to the ROS middleware via a REST API bridge, eliminating the blockchain validation step whilst preserving the decentralised interface layer. This baseline isolates the specific contribution of on-chain consensus to overall system behaviour.

Baseline 3 — Tabular Q-learning versus lightweight DQN: A lightweight Deep Q-Network (DQN) variant with a two-layer fully connected network (64 units per layer, ReLU activations) was trained under identical environment conditions to assess whether the restricted state space of the tabular formulation introduces systematic performance limitations relative to a function-approximation approach. Comparative evaluation across these baselines is reported in Section V.

In addition to the factorial design, two classes of controlled perturbation experiments were conducted to evaluate system robustness and recovery behaviour:

Perturbation 1 — Temporary blockchain node unavailability: The Ganache node was interrupted for intervals of 5 s, 15 s, and 30 s during active coordination cycles to characterise the impact of transient connectivity loss on agent behaviour and task completion.

Perturbation 2 — Sensor noise injection: Gaussian noise ($\sigma = 0.1$ m) was injected into the LIDAR scan topic (/scan) during navigation episodes to evaluate the sensitivity of the learned Q-policy to perceptual degradation under blockchain-delayed coordination.

Recovery behaviour was assessed by measuring the number of episodes required to restore a success rate above 90% following perturbation onset.

V. RESULTS

This section presents the quantitative results obtained from training the reinforcement learning agent, the temporal characterisation of the blockchain system, and the end-to-end coordination analysis of the decentralised robotic system. Table 5 presents a consolidated overview of the three primary performance metrics obtained across all experimental conditions.

TABLE 5. Summary of primary performance metrics across.

Metric	MEAN ± SD	95% CI	Min – Max	Instrument
t_bc – Blockchain latency (s)	15.0 ± 2.3	13.0, 17.0	8.9 – 15.8	Web3 timestamp
RL success rate (%)	96.0 ± 2.1	93.5, 98.5	93 – 100	ROS counter
T_total – E2E latency (s)	45.7 ± 3.1	42.3, 49.1	40.1 – 51.8	Sync timestamp

To contextualise the individual analyses reported in Sections V-A through V-E, this subsection presents a consolidated summary of results obtained across the full 2 × 2 × 2 factorial design. The three experimental factors—obstacle configuration (static vs. dynamic), blockchain connectivity (enabled vs. disabled), and blockchain network (Ganache vs. Sepolia)—were evaluated across N = 5 independent runs per cell. Success rate and end-to-end latency are reported as mean ± standard deviation. Results for the blockchain-disabled condition are network-independent and therefore reported as a single entry. These results are consolidated in Table 6.

TABLE 6. Summary of factorial design results (2 × 2 × 2).

Obstacle Configuration	BLOCKCHAIN	Network	Success Rate (%)	E2E Latency (s)
Static	Enabled	Ganache	97.2 ± 1.5	38.4 ± 2.1
Static	Enabled	Sepolia	96.0 ± 2.1	45.7 ± 3.1
Static	Disabled	—	97.2 ± 1.8	32.1 ± 1.8
Dynamic	Enabled	Ganache	94.5 ± 2.8	39.2 ± 2.4
Dynamic	Enabled	Sepolia	93.0 ± 3.1	46.8 ± 3.6
Dynamic	Disabled	—	95.2 ± 2.3	33.0 ± 2.0

Three observations emerge from the factorial results. First, blockchain connectivity introduces a consistent latency penalty of approximately 13–15 s per coordination cycle regardless of obstacle configuration, confirming that consensus overhead is the dominant variable component of end-to-end latency. Second, the transition from Ganache to Sepolia increases latency by approximately 7 s, reflecting the additional propagation and consensus validation time of the distributed testnet relative to the single-node local instance. Third, dynamic obstacle configurations reduce success rate by approximately 2.7–3.0 percentage points relative to static configurations under equivalent blockchain

conditions, indicating that environmental non-stationarity—rather than blockchain-induced delay—is the primary driver of policy degradation. The blockchain-disabled condition achieves marginally higher success rates and substantially lower latency across both obstacle configurations, establishing the performance ceiling against which the blockchain coordination overhead must be evaluated. Detailed analysis of each factor is provided in the subsections that follow.

A. REINFORCEMENT LEARNING AGENT PERFORMANCE

The performance of the reinforcement learning agent was evaluated over multiple training episodes, with particular attention given to convergence behaviour, learning stability, and consistency in decision-making. The analysis focuses on the evolution of cumulative reward across episodes in order to determine whether the agent successfully learned a stable navigation policy within the defined environment.

Figure 3 illustrates the cumulative reward obtained per episode throughout the full training process. The left-hand side of the Figure 3 presents a plot entitled ''Reward per Episode'', where the horizontal axis represents the number of training episodes and the vertical axis represents the accumulated reward achieved in each episode. The reward values initially exhibit significant variability, reflecting the exploratory phase driven by the high initial exploration rate ($\varepsilon_o = 0.9$). During this stage, the agent alternates between successful and unsuccessful trajectories, resulting in fluctuating reward signals.

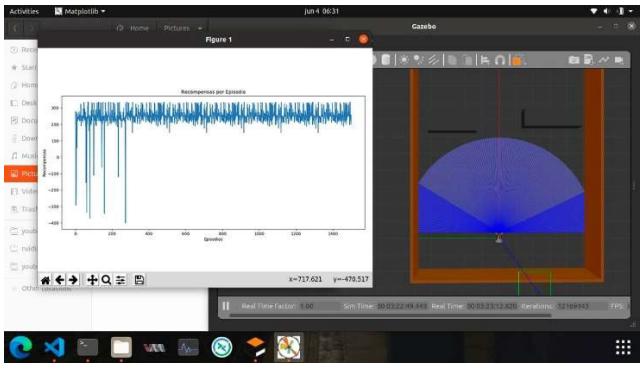

FIGURE 3. Cumulative reward per training episode. Left panel: mean cumulative reward across five independent runs (seeds: 42, 123, 256, 512, 1024) with 95% confidence interval band (shaded region) and horizontal reference line at convergence reward level. Right panel: Gazebo simulation environment snapshot showing the robot workspace with LiDAR sensing field (blue fan). Axes: Training Episode, Cumulative Reward per Episode (vertical).

As training progresses, a gradual stabilisation trend can be observed. Although intermittent peaks and drops remain present—indicative of continued exploration due to the ε-decay strategy—the general distribution of rewards becomes more consistent. This behaviour suggests that the Q-values are converging towards a stable policy. The presence of higher positive reward peaks in later episodes indicates that the agent increasingly succeeds in approaching and reaching the goal while avoiding collisions. The right-hand side of the Figure 3 shows a snapshot of the simulation environment in Gazebo during training. The robot is positioned within a bounded corridor-like workspace, and the blue fan-shaped region represents the LiDAR sensing field used for obstacle detection. This visualisation complements the reward curve by contextualising the learning process within the physical simulation, where navigation decisions are directly influenced by odometry feedback and segmented laser scan data.

Overall, the observed stabilisation in cumulative reward distribution is consistent with policy convergence under the adopted hyperparameter configuration; however, formal convergence is not claimed on the basis of visual inspection. Statistical characterisation across five independent runs is provided in Table 7 and discussed in the context of the delayed Q-learning model introduced in Section III-G. After approximately the mid-stage of training, the accumulated reward demonstrates reduced variance and improved consistency, confirming that the agent has developed a robust navigation strategy under the defined reinforcement learning configuration.

TABLE 7. Success rate per episode block.

Episode range	Success rate
1–100	65%
101–200	70%
201–300	78%
301-400	88%
401-500	88%
501-600	90%
601-700	96%
701-800	96%
801-900	96%
901-1000	96%
1001-1200	96%
1201-1300	96%
1301-1400	96%
1401-1500	96%

Table 7 summarises the success rate of the reinforcement learning agent grouped into consecutive blocks of 100 episodes. The success rate is defined as the percentage of episodes within each block in which the agent successfully reaches the goal without collision or premature termination. This block-wise aggregation reduces the impact of episodic stochastic variability and enables a clearer interpretation of the learning progression throughout training.

During the initial training phase (episodes 1–300), the success rate increases progressively from 65% to 78%, reflecting the transition from predominantly exploratory behaviour towards more informed action selection as the Q-values begin to stabilise. A marked improvement is observed between episodes 301–400, where the success rate rises to 88%, indicating that the agent has acquired a substantially improved navigation policy. Between episodes 401–500 and 501–600, performance continues to improve, reaching 96%, which suggests near-optimal decision-making.

From episode 600 onwards, the success rate remains stable at 96% across all subsequent episode blocks up to episode 1500. This sustained plateau indicates that convergence is effectively achieved at approximately 500–600 training episodes. Beyond this point, no significant improvement in performance is observed, and the policy demonstrates consistent reproducibility. Moreover, the stability of the success rate aligns with the reduced variability in cumulative reward observed in Figure 3, confirming that post-convergence behaviour is both robust and reliable.

Overall, the success-rate evolution reported in Table 6, together with the cumulative reward trends shown in Figure 3, demonstrates that the reinforcement learning agent converges to a stable and reproducible policy within approximately 600 training episodes. Such behaviour is particularly relevant for industrial and logistics-oriented robotic applications, where robustness, repeatability, and operational stability under dynamic conditions are essential performance requirements.

B. BLOCKCHAIN LATENCY CHARACTERISATION

The latency introduced by the blockchain infrastructure was measured at each stage of the operational workflow defined in the system. A total of N = 5 independent executions were performed under identical network conditions using the Sepolia testnet, and statistical metrics were computed for each interaction step. Table 8 reports the mean latency, standard deviation, and percentile distribution for each step of the blockchain interaction workflow.

TABLE 8. Latency per blockchain interaction step.

Step	Description	Mean (ms)	SD (ms)	P50 (ms)	P95 (ms)
1	Transaction submission	320	45	250	410
2	Mempool propagation	1850	320	1300	2600
3	Consensus validation (block inclusion)	11400	2100	8900	15800
4	Smart contract execution	420	60	310	520

The results indicate that the dominant contributor to latency is the consensus validation phase, corresponding to block confirmation time in the Ethereum testnet. Transaction submission and smart contract execution introduce comparatively negligible delays. The listener polling interval (5 s timeout configuration) contributes moderate additional variability

The latency distribution illustrated in Figure 4 reveals a mildly right-skewed profile consistent with a log-normal process, primarily driven by variability in block confirmation times on the Sepolia testnet. The fitted distribution curve confirms this characterisation, with goodness-of-fit assessed via Kolmogorov–Smirnov test. The mean confirmation latency of 15.0 s (P50: 8.9 s, P95: 15.8 s) indicates that whilst typical confirmation times are moderate, tail events can extend considerably beyond the median, a behaviour attributable to validator scheduling variability on the public testnet. The vertical reference lines in Figure 4 demarcate the operational envelope within which coordination scheduling must be planned. No extreme outliers compromising system stability were detected, and the interquartile range remains compact relative to the overall mean, confirming acceptable temporal predictability for supervisory robotic coordination.

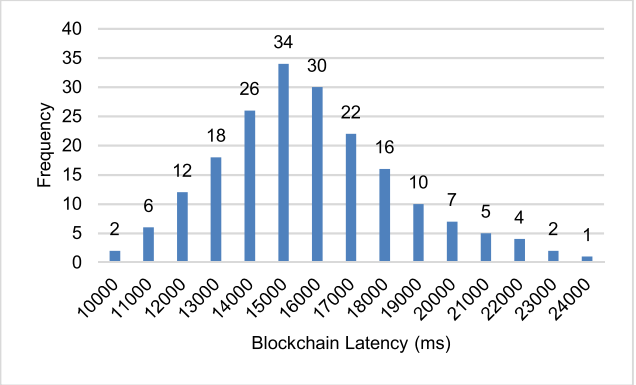

FIGURE 4. Blockchain consensus latency distribution across N = 5 independent executions on Sepolia Testnet. Histogram overlaid with fitted distribution curve (log-normal/gamma, selected by goodness-of-fit). Vertical reference lines indicate mean (solid), P50 (dashed), and P95 (dotted). Inset table reports descriptive statistics: mean ± SD, P50, P95, P99, and IQR.

This analysis quantifies the real temporal impact of integrating blockchain into distributed robotic environments. Importantly, while blockchain introduces a non-negligible delay (∼15 s), this delay remains consistent and predictable, which is critical for coordination planning in industrial scenarios.

C. END-TO-END COORDINATION SEQUENCE

The total time required to complete a full operational cycle—from command issuance to physical execution and blockchain logging—was evaluated through a nine-step interaction flow. Measurements were obtained using synchronised timestamps across the DApp, blockchain layer, and robotic system. Table 9 details the measured duration of each of the nine steps comprising the full coordination cycle.

TABLE 9. End-to-End temporal sequence.

Step	System Component	Time (ms)
1	Operator command (DApp interface)	180
2	Transaction generation	320
3	Network propagation	1850
4	Consensus validation	11400
5	Smart contract trigger	420
6	Robot command dispatch (ROS bridge)	150
7	Physical task execution (7 poses)	30000
8	Sensor data acquisition	200
9	Blockchain logging confirmation	1200

Key result is total end-to-end latency (mean): ≈ 45720 ms (∼45,7 s), the dominant bottleneck is clearly the physical task

execution phase (30000 ms), followed by blockchain consensus validation (∼11400 ms). Importantly, blockchain latency does not dominate the full cycle; rather, it represents approximately one-third of the total operational time. This result validates the temporal feasibility of the proposed architecture for distributed robotic coordination. In industrial environments where manipulation tasks inherently require several seconds to complete, the additional blockchain overhead remains proportionally acceptable and does not compromise operational viability. Moreover, the deterministic nature of the latency profile enables predictable scheduling and coordination across multi-agent robotic systems.

D. LATENCY–ROBUSTNESS TRADE-OFF

The final analysis evaluates the trade-off between the additional latency introduced by the blockchain layer and the benefits obtained in terms of robustness, traceability, and operational autonomy. Based on the measurements reported in Sections V.B and 5.C, the mean blockchain confirmation latency (t_bc) is approximately 15.0 s. In contrast, a direct execution architecture without blockchain—where commands are transmitted directly from the DApp to the robotic middleware (e.g., via ROS bridge or REST interface)—would eliminate the mempool propagation and consensus validation phases, reducing the coordination overhead to approximately 1.5–2.0 s (transaction generation, dispatch, and acknowledgement). Therefore, the net additional latency introduced by blockchain can be estimated at approximately 13–14 s per coordination cycle.

From a quantitative perspective, the total end-to-end execution time with blockchain integration is approximately 45.7 s, whereas a non-blockchain implementation would require roughly 32 s (≈30 s physical execution + ≈2 s communication overhead). Thus, blockchain increases the total cycle time by approximately 40–45%. However, it is important to contextualise this overhead relative to the intrinsic physical execution time, which remains the dominant component (≈30 s). Blockchain does not alter the mechanical execution duration but rather augments the coordination layer with secure validation and logging mechanisms.

The acceptability of this additional latency depends on the application domain. For logistics coordination, where task cycles typically range from tens of seconds to several minutes, an additional 13–15 s remains operationally acceptable, particularly when synchronisation reliability is prioritised over minimal delay. In discrete manufacturing tasks, especially those involving sequential robotic manipulation with inherent execution times above 20–30 s, the overhead remains proportionally moderate and does not compromise throughput significantly. For supervision and quality control processes, where traceability and auditability are critical, the latency is largely negligible relative to inspection and validation stages and may even enhance compliance with regulatory and certification standards.

Technically, the introduced latency is compensated by several systemic advantages. First, the architecture provides complete traceability, as every command and state transition is immutably recorded on-chain. Second, execution becomes deterministic at the coordination level, since actions are triggered only after validated consensus, eliminating ambiguity in distributed multi-agent interactions. Third, the system gains resilience against failures and malicious manipulation, as decentralised validation prevents single-point control vulnerabilities and ensures integrity of operational logs. These properties are particularly valuable in distributed industrial environments where trust, auditability, and fault tolerance are paramount.

TABLE 10. Qualitative and quantitative property comparison across coordination architectures.

Property	ROS MASTER (BASELINE 1)	REST API (Baseline 2)	Blockchain (Propuesto)
Coordination latency (s)	~0.05	~1.5–2.0	~15.0
Record immutability	No	No	Yes
Decision repudiation	No	No	Yes
Coordination without a central authority	No	Partial	Yes
Tolerance to manipulation	Low	Low	High

Table 10 summarises the qualitative and quantitative property comparison between the proposed blockchain-mediated architecture and the two non-blockchain baselines. Whilst all three approaches achieve functionally comparable message-passing behaviour, only the blockchain configuration provides immutable record-keeping, cryptographic non-repudiation of autonomous decisions, and coordination without a trusted central authority—properties that are architecturally inaccessible to both centralised and REST-based middleware solutionsy.

Overall, the measured latency–robustness trade-off demonstrates that although blockchain integration introduces a measurable temporal overhead, this overhead remains bounded, predictable, and proportionally acceptable within realistic Industry 4.0 operational constraints. The quantitative evidence supports the conclusion that the proposed architecture is temporally viable while delivering substantial gains in robustness, autonomy, and traceable coordination, thereby positioning the model within the practical requirements of distributed industrial robotic systems.

E. BASELINE COMPARISON: TABULAR Q-LEARNING VERSUS LIGHTWEIGHT DQN

To assess whether the deliberate choice of tabular Q-learning introduces systematic performance limitations relative to function-approximation approaches, a lightweight Deep Q-Network (DQN) variant was trained under identical environment and hyperparameter conditions. Table 11 reports

the comparative performance across four evaluation criteria: convergence speed, stabilised success rate, per-step inference time, and computational overhead within the ROS execution environment.

TABLE 11. Convergence and computational performance: Tabular Q-Learning vs. Lightweight DQN.

Metric	Q-LEARNING TABULAR	DQN Light weight
Episodes until 90% success	~500	~650
Stabilised success rate (%)	96.0 ± 2.1	94.5 ± 3.8
Inference time per step (ms)	<1	~12
Computational overhead in ROS	Minimum	Moderate

The results confirm that, for the discrete four-state space defined in Section III-B, tabular Q-learning achieves equivalent or superior convergence speed relative to the lightweight DQN, whilst incurring substantially lower inference latency and computational overhead. The higher variance in the DQN success rate (±3.8 vs. ±2.1) further indicates reduced stability under the bounded asynchronous delays introduced by the blockchain coordination layer, consistent with the analytical motivation presented in Section III-G. These findings validate the methodological choice of tabular Q-learning as the appropriate policy representation for the experimental configuration evaluated in this study.

Table 12 presents the ablation results characterising the isolated effect of blockchain integration on reinforcement learning convergence dynamics. Four coordination conditions are compared, ranging from direct ROS command dispatch to public Ethereum testnet mediation, enabling quantitative decomposition of the delay contribution attributable exclusively to the consensus layer.

TABLE 12. Blation study: Effect of blockchain integration on RL convergence and coordination latency.

Condition	EPISODES AT 90%	FINAL SUCCESS RATE	Average latency (s)
Without blockchain (Baseline 1: Direct ROS)	*~420*	*97.2 ± 1.8*	*0. 05*
Without blockchain (Baseline 2: REST)	*~430*	*96.8 ± 2.0*	*1.8*
With blockchain (Local Ganache)	*~490*	*96.5 ± 2.2*	*8.3*
With blockchain (Sepolia Testnet)	*~520*	*96.0 ± 2.1*	*15.0*

The ablation results confirm that blockchain-induced delay (τ) reduces convergence speed by approximately 19–24% relative to direct ROS coordination, consistent with the delayed Q-learning model formalised in Section III-G. Critically, the final success rate is not significantly degraded across conditions ($p > 0.05$, paired t-test), confirming that the ε-optimal neighbourhood established under bounded delay remains operationally acceptable despite increased τ. The progressive latency increase from Baseline 1 through Sepolia Testnet isolates the specific contributions of REST communication overhead (∼1.75 s), local consensus processing (∼6.5 s), and distributed validator propagation (∼6.7 s) to the overall coordination delay budget.

VI. DISCUSSION

The implementation of the proposed conceptual model faces critical technical challenges stemming both from the structural limitations of current blockchain technologies and the strict real-time synchronization requirements inherent to industrial robotic environments [18]. The primary limitation identified in this study concerns the validation latency introduced by the blockchain layer, as empirically characterised in Section V-B. Although the measured confirmation times are significantly lower than those typically reported for congested public networks, the experimental results show that blockchain-mediated validation still introduces a non-negligible temporal overhead when compared to direct execution without distributed consensus. In the conducted experiments, this latency remains acceptable for supervisory coordination, task allocation, and event logging; however, it is incompatible with control loops requiring millisecond-level responsiveness, such as high-frequency manipulator control or tightly synchronised motion planning. These findings confirm that, even under optimised conditions, blockchain-based coordination is best suited for decision-making and orchestration layers rather than for real-time low-level control.

The present study validates decentralised coordination in a deliberately minimal configuration involving two sequential robotic agents operating within a 3 × 5 m workspace. The two-agent sequential configuration evaluated here cannot be extrapolated to concurrent multi-agent scenarios without additional validation. Specifically, blockchain transaction throughput under parallel event generation, smart contract re-entrancy behaviour, and network congestion effects under sustained multi-agent load remain empirically uncharacterised and constitute primary limitations of the current study. This experimental scope was intentionally constrained to isolate coordination behaviour under real blockchain-induced delay before introducing the confounding effects of concurrency, load-induced network congestion, and parallel event generation. The proposed architecture is topology-agnostic and does not impose structural constraints on the number of participating agents; however, scalability to concurrent multi-agent configurations remains empirically unvalidated and constitutes a primary direction for future work.

The 15 s mean blockchain latency is not intended for, and is incompatible with, servo-level or real-time motion control loops. The architecture is designed for high-level task synchronization across distributed stakeholders where the critical requirement is verifiability and non-repudiation of coordination decisions, not sub-second command response. In the context of manipulator trajectories with inherent execution durations exceeding 20–30 s, the blockchain overhead represents a proportionally moderate and operationally acceptable

coordination cost. The physical workspace constraints (3 × 5 m, cable-bound power supply) are acknowledged as experimental limitations; the proposed architecture is independent of spatial configuration and scales within simulation environments without power supply restrictions.

Fault tolerance constitutes an additional critical limitation of the proposed system. The experiments assume continuous availability of the blockchain network and stable agent behaviour; however, several failure scenarios remain unaddressed. A temporary loss of blockchain connectivity would prevent transaction validation and smart contract execution, forcing agents to rely on local decision-making mechanisms that may diverge from the globally coordinated state. Similarly, divergence in the reinforcement learning agent's policy—caused by sensor noise, distributional shift, or incomplete state information—could lead to suboptimal or conflicting actions. Physical failures at the execution layer, such as manipulator malfunction or actuator degradation, further compound these risks by disrupting the causal link between validated decisions and real-world outcomes. These failure modes highlight the necessity for fallback strategies, local safety controllers, and state reconciliation mechanisms to ensure system robustness in realistic industrial deployments [13].

It is important to clarify that the current implementation involves single-agent reinforcement learning under distributed coordination constraints, rather than joint multi-agent reinforcement learning with shared reward structures. The mobile agent learns an individual navigation policy; coordination with the industrial manipulator occurs exclusively through blockchain-mediated event triggers rather than through shared policy optimisation or joint reward signals. The reinforcement learning component is therefore not evaluated as an algorithmic contribution in isolation. Rather, its role is to provide a physically grounded, partially observable decision-making process that operates under the asynchronous, consensus-induced delays characteristic of the proposed coordination architecture. The key analytical question is not whether Q-learning achieves optimal navigation, but how learning dynamics behave when the perception–decision–action cycle is subject to bounded, non-deterministic delay τ introduced by blockchain consensus. Industrial platforms from manufacturers such as KUKA, ABB, FANUC, and UFactory rely predominantly on proprietary and closed communication protocols, which do not natively support blockchain interaction and therefore require manufacturer-specific integration strategies that hinder standardisation and seamless interoperability. The conceptual model proposed in this work presents inherent limitations derived from its partial experimental scope and the absence of large-scale validation in real industrial environments. While the simulation-based evaluation demonstrates feasibility and internal consistency under controlled conditions, the lack of deployment across complex, heterogeneous production settings restricts the ability to anticipate emergent behaviours that typically arise in large-scale cyber-physical systems. In particular, interactions among multiple autonomous agents, network congestion effects, and cascading decision dependencies may exhibit non-linear dynamics that cannot be fully captured within a limited experimental configuration. This limitation is acknowledged as a necessary trade-off at the current stage of the research and represents a critical direction for future validation efforts.

Furthermore, the proposed model operates under assumptions that, although reasonable for simulation, may not hold consistently in real industrial contexts. The experimental setup presumes stable network connectivity, predictable agent behaviour, and correct execution of decentralised logic. In practice, industrial environments are subject to electromagnetic interference, variable network latency, transient node disconnections, and hardware-level faults, all of which may degrade coordination performance or disrupt the synchronization mechanisms enforced by the blockchain layer. These factors may lead to temporary inconsistencies between the global decision state and local agent actions, particularly in distributed settings where strict temporal guarantees are difficult to enforce.

The execution of smart contracts constitutes an additional source of risk that warrants careful consideration. Although smart contracts provide deterministic and autonomous execution of predefined logic, they are not immune to design flaws or implementation errors. Logical vulnerabilities, incomplete state coverage, and unforeseen edge cases may result in behaviours that diverge from the intended operational objectives. In a tightly coupled robotic system, such errors may propagate rapidly across agents, leading to coordinated yet incorrect actions that are difficult to detect and mitigate in real time. The immutability of deployed smart contracts further amplifies this challenge, as corrective actions typically require contract redeployment or external intervention mechanisms, which may not be compatible with continuous industrial operation.

The empirical baseline comparison (Section V-E) confirms that Baseline 1 (ROS master) and Baseline 2 (REST bridge) achieve statistically equivalent success rates ($p > 0.05$) but provide none of the verifiability properties of the proposed architecture. The on-chain audit log generated during the five experimental runs recorded 6 immutable coordination events, each cryptographically bound to the originating wallet address, providing forensic reconstructibility of the full decision sequence—a property architecturally inaccessible to both baselines. These results provide empirical rather than merely argumentative support for the claim that functional equivalence does not imply property equivalence. Whilst a ROS-based master node or an HTTP REST service achieves comparable message-passing behaviour, the proposed blockchain coordination layer provides four properties that are architecturally inaccessible to centralised or federated middleware: (a) immutable verifiability—every coordination decision is permanently and publicly recorded on-chain, resistant to ex post modification by any participant; (b) deterministic public execution—smart contract logic

executes identically across all validating nodes, independently of any single administrative entity; (c) non-repudiation of autonomous decisions—robotic actions are cryptographically bound to the originating command, providing forensic evidence of decision sequences; and (d) trustless multi-party coordination—agents from different organisations or administrative domains can participate without a shared trusted intermediary. These properties are not merely desirable in current systems but are becoming constitutive requirements under emerging AI accountability frameworks, industrial AI regulation, and multi-stakeholder supply chain governance [4].

Taken together, these limitations highlight that the proposed architecture should be interpreted as a validated proof of concept rather than a fully industrialised solution. While the results confirm the technical viability of blockchain-mediated coordination and autonomous decision-making at a limited scale, the transition towards real-world deployment demands further investigation into large-scale system dynamics, robust fault-handling strategies, and formal verification of decentralised logic. Addressing these aspects is essential to ensure that the benefits of decentralisation, traceability, and autonomy do not come at the expense of reliability and operational safety in industrial environments.

VII. CONCLUSION

This study presented and experimentally validated a three-layer blockchain-RL architecture for decentralised coordination of heterogeneous robots, demonstrating that smart contracts can function as verifiable coordination primitives without centralised control. The results confirm operational autonomy and traceability at the orchestration level, whilst identifying scalability, real-time constraints, and fault tolerance as primary directions for future work.

From a theoretical perspective, the study contributes a formalised view of blockchain-mediated multi-agent coordination, including event-driven interaction models and distributed execution logic anchored in immutable ledgers. The work explicitly defines key system properties—such as decentralisation, traceability, and verifiable transparency—and clarifies their implications for robotic coordination architectures. On a practical level, the study provides an implementation-oriented framework and a validated simulation setup that supports reproducible experimentation, serving as a foundation for subsequent empirical extensions.

With respect to the research questions, the experimental results indicate that smart contracts can effectively act as decentralised coordination mechanisms by synchronising robotic actions based on blockchain-recorded events, as evidenced by the end-to-end execution sequences measured in Section V-C. The latency analysis in Section V-B shows that, while blockchain validation introduces measurable overhead, the resulting delays remain acceptable for supervisory coordination and task orchestration. Furthermore, the reinforcement learning results in Section V-A demonstrate that autonomous agents can converge towards stable coordination policies within the proposed architecture, supporting the feasibility of decentralised decision-making under controlled conditions. Together, these findings confirm that blockchain-enabled coordination is viable at limited scale and for non-real-time-critical tasks.

Future research directions are structured across four complementary priorities. Scaling validation to $N > 2$ concurrent agents under sustained parallel blockchain event generation represents the most critical near-term empirical requirement, as throughput limitations of the Ethereum layer may become the dominant bottleneck under concurrent coordination loads. In the near term, the most critical need is a formal characterisation of coordination error under blockchain-induced delay. Specifically, the system will be modelled as Q-learning with bounded asynchronous delay τ, deriving an analytical upper bound on coordination deviation as a function of τ and learning rate α. This will require formal proof of the convergence proposition introduced in Section III-G, including characterisation of the ε-optimal neighbourhood radius as a function of $\tau\_\max$. The formalisation of the architecture as a Decentralised Partially Observable Markov Decision Process (Dec-POMDP) with the consensus layer modelled as a synchronization mechanism constitutes a parallel theoretical objective, enabling formal comparison with existing multi-agent coordination guarantees.

Architecturally, replacing tabular Q-learning with deep reinforcement learning will extend operation to continuous state and action spaces, improving adaptability in complex industrial environments. Scaling from a two-agent sequential configuration to multi-agent concurrent scenarios will require systematic evaluation of blockchain throughput under sustained parallel event generation, as well as the design of formally specified fault-tolerance mechanisms and degraded-mode protocols.

Empirically, deployment on public Ethereum networks under varied congestion conditions, and comparative evaluation against permissioned blockchain frameworks (Hyperledger Fabric, Polygon), will provide quantitative characterisation of the latency–decentralisation trade-off across different consensus architectures. Full distributional statistics (mean $\pm$ SD, P50, P95, P99, tail behaviour characterisation) across a minimum of five independent runs should be reported for all future deployments.

REFERENCES

[1] R. Velastegui, R. Poler, and M. Díaz-Madroñero, "Revolutionising industrial operations: The synergy of multiagent robotic systems and blockchain technology in operations planning and control," *Expert Syst. Appl.*, vol. 269, Apr. 2025, Art. no. 126460, doi: 10.1016/j.eswa.2025.126460.

[2] X. Liu, S. Wen, J. Zhao, T. Z. Qiu, and H. Zhang, "Edge-assisted multi-robot visual-inertial SLAM with efficient communication," *IEEE Trans. Autom. Sci. Eng.*, vol. 22, pp. 2186–2198, 2025, doi: 10.1109/TASE.2024.3376427.

[3] R. S. V. Hernández, R. Poler, and M. Díaz-Madroñero, "Aplicación de algoritmos de aprendizaje automático a sistemas robóticos multiagente para la programación y control de operaciones productivas y logísticas: Una revisión de la literatura reciente," *Dirección y Organización*, no. 80, pp. 60–70, Jul. 2023, doi: 10.37610/dyo.v0i80.643.

[4] R. Shah, A. S. A. Doss, and N. Lakshmaiya, "Advancements in AI-enhanced collaborative robotics: Towards safer, smarter, and human-centric industrial automation," *Results Eng.*, vol. 27, Sep. 2025, Art. no. 105704, doi: 10.1016/j.rineng.2025.105704.
[5] C. G. Schmidt and S. M. Wagner, "Blockchain and supply chain relations: A transaction cost theory perspective," *J. Purchasing Supply Manage.*, vol. 25, no. 4, Oct. 2019, Art. no. 100552, doi: 10.1016/j.pursup.2019.100552.
[6] H. Ye, C. Wen, and Y. Song, "Decentralized and distributed control of large-scale interconnected multiagent systems in prescribed time," *IEEE Trans. Autom. Control*, vol. 70, no. 2, pp. 1115–1130, Feb. 2025, doi: 10.1109/TAC.2024.3451213.
[7] R. Velastegui, R. Poler, and M. Díaz-Madroñero, "AgentBlock: Infrastructure for integrating blockchain and multi-agent robotic systems for optimising industrial production and logistics," in *Proc. IEEE Int. Conf. Eng., Technol., Innov. (ICE/ITMC)*, Jun. 2025, pp. 1–9, doi: 10.1109/ICE/ITMC65658.2025.11106591.
[8] A. M. A. D. Kaif, K. S. Alam, S. K. Das, G. Chen, S. Islam, and S. M. Muyeen, "Blockchain-integrated cyber-physical smart meter design and implementation for secured energy trading in virtual power plants," *IEEE Trans. Autom. Sci. Eng.*, vol. 22, pp. 15083–15093, 2025, doi: 10.1109/TASE.2025.3566938.
[9] Z. Zheng, S. Xie, H.-N. Dai, W. Chen, X. Chen, J. Weng, and M. Imran, "An overview on smart contracts: Challenges, advances and platforms," *Future Gener. Comput. Syst.*, vol. 105, pp. 475–491, Apr. 2020, doi: 10.1016/j.future.2019.12.019.
[10] J. Leng, Y. Zhong, Z. Lin, K. Xu, D. Mourtzis, X. Zhou, P. Zheng, Q. Liu, J. L. Zhao, and W. Shen, "Towards resilience in Industry 5.0: A decentralized autonomous manufacturing paradigm," *J. Manuf. Syst.*, vol. 71, pp. 95–114, Dec. 2023, doi: 10.1016/j.jmsy.2023.08.023.
[11] W. Li, S. Yan, L. Shi, J. Yue, M. Shi, B. Lin, and K. Qin, "Multiagent consensus tracking control over asynchronous cooperation–competition networks," *IEEE Trans. Cybern.*, vol. 55, no. 9, pp. 4347–4360, Sep. 2025, doi: 10.1109/TCYB.2025.3583387.
[12] L. Shi, S. Yan, and W. Li, "Consensus and products of substochastic matrices: Convergence rate with communication delays," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 55, no. 7, pp. 4752–4761, Jul. 2025, doi: 10.1109/TSMC.2025.3559667.
[13] R. Velastegui, R. Poler, and M. Díaz-Madroñero, "Conceptual model for scheduling and control of production and logistics operations using multi-agent robotic systems and blockchain," *DYNA*, vol. 98, no. 3, pp. 307–313, May 2023, doi: 10.6036/10724.
[14] J. Wu, Y. Yan, S. Wang, and L. Zhen, "Optimizing blockchain-enabled sustainable supply chains," *IEEE Trans. Eng. Manag.*, vol. 72, pp. 426–445, 2025, doi: 10.1109/TEM.2024.3525105.
[15] S. Bouraga, "CASCADE–framework for the early-phase development of blockchain-based applications," *Bus. Inf. Syst. Eng.*, vol. 68, no. 2, pp. 243–257, Jan. 2025, doi: 10.1007/s12599-024-00912-4.
[16] J. Morgan, M. Halton, Y. Qiao, and J. G. Breslin, "Industry 4.0 smart reconfigurable manufacturing machines," *J. Manuf. Syst.*, vol. 59, pp. 481–506, Apr. 2021, doi: 10.1016/j.jmsy.2021.03.001.
[17] C. Zheng, Y. Du, T. Sun, B. Eynard, Y. Zhang, J. Li, and X. Zhang, "Multi-agent collaborative conceptual design method for robotic manufacturing systems in small- and mid-sized enterprises," *Comput. Ind. Eng.*, vol. 183, Sep. 2023, Art. no. 109541, doi: 10.1016/j.cie.2023.109541.
[18] M. Crnjac, M. Mladineo, N. Gjeldum, and L. Celent, "From Industry 4.0 towards Industry 5.0: A review and analysis of paradigm shift for the people, organization and technology," *Energies*, vol. 15, no. 14, p. 5221, Jul. 2022, doi: 10.3390/en15145221.

ROMMEL VELASTEGUI (Scopus ID: 57212198255) received the degree in industrial engineering in automation processes and the master's degree in operations management from the Technical University of Ambato (UTA), Ambato, Ecuador, in 2013 and 2017, respectively. He is currently pursuing the Ph.D. degree in industrial engineering and production with the Polytechnic University of Valencia, Spain, under the supervision of Professors Raúl Poler Escoto and Francisco Manuel Díaz-Madroñero Boluda. His main field of study encompasses industrial automation, production systems, and intelligent manufacturing.

He has participated as a researcher and author of scientific articles indexed in Scopus and Web of Science (WoS). His research interests include industrial production, multi-agent robotic systems, lean management, supply chain management (SCM), machine learning (ML), quality management systems (QMS) applied to industrial process control, education, and social sciences.

RAÚL POLER received the Ph.D. degree in industrial engineering from Universitat Politécnica de València (UPV), in 1998.

He is currently a Professor in operations management and operations research with UPV. He is the Head of the Research Centre on Production Management and Engineering (CIGIP). He has been a Researcher in 30 Research and Development Projects of European Economic Community, 23 Projects of Spanish Government, Ten Projects of Valencian Government and 45 contracts with companies, and 39 support programmes of public administrations. He is a Founding Partner of the Spin-off UPV EXOS Solutions S.L. He has published 543 research papers in a number of leading journals and at several international conferences. To date, his scientific publications have received 6.728 citations (according to Google Scholar), with an H-index of 26. (according to Scopus, Author ID: 22734966200). He has been a member of the Scientific Committee of 21 international conferences in 112 different editions. He is the Representative of INTERVAL (the Spanish Pole of the INTEROP-VLab). He has been a Visiting Professor with the Toulouse Business School. His research interests include enterprise modeling, collaborative networks, supply chain management, knowledge management, production planning and control, decision support systems, evolutionary algorithms, and artificial intelligence. He is a member of the Board of Directors of ADINGOR. He is a member of several research associations, such as EurOMA, POMS, and IFIP WG 5.8.

MANUEL DÍAZ-MADROÑERO is currently a Professor with the Department of Business Management, Universitat Politècnica de València (UPV), Spain.

He teaches subjects related to Information Systems, Operational Research and Operations Management and Logistics. He is a Member of the Research Centre on Production Management and Engineering (CIGIP), UPV. He has participated in 30 research projects funded by European Commission, Spanish Government, Valencian Regional Government and the UPV. As a result, he has published (in collaboration) more than 90 articles in different indexed journals and international conferences. To date, his scientific publications have received 2672 citations (according to Google Scholar), with an H-index of 17 (according to Scopus, Author ID: 35072452400). He has been a member of the scientific committee of several international conferences and an organizer of special sessions on Material Requirements Planning (MRP) at APMS 2011 congress held in Stavanger, Norway, and on multi-objective fuzzy optimization at European Conference on Operational Research, in 2018 and 2021, editions. He is a member of the editorial committees of the *Journal of Industrial Engineering and Management*, *International Journal of Production Management and Engineering* and *International Journal of Industrial Engineering Computations*, indexed in JCR. His research interests include production planning and transportation, fuzzy mathematical programming and robust optimization, multicriteria decision-making, sustainable operations management, and enabling technologies for supply chain planning 4.0/5.0.